

Multi-Source Candidate Data Transformer - Technical Design

Eightfold Engineering Intern Assignment (Jul-Dec 2026) | one-page design, no code

Problem Goal

The transformer should take messy candidate information from multiple places and produce one clean, canonical JSON profile per person. I prioritize correctness over filling every blank: a missing value is safer than a confident but wrong value. The design handles a structured recruiter CSV plus GitHub profile data and recruiter notes text as unstructured/semi-structured evidence.

Pipeline Breakdown

Detect -> Extract -> Normalize -> Merge -> Confidence -> Project -> Validate. Detect finds which inputs are present. Extract converts CSV, GitHub, and notes into one internal evidence format: {field, value, source, method, structured, confidence}. Normalize cleans phones, countries, dates, skills, names, and emails. Merge groups records into candidates, records conflicts, and keeps provenance. Confidence scores explain trust. Project reshapes the canonical profile using runtime config. Validate checks the requested output before writing JSON.

Canonical Output Schema And Formats

Field	Shape / Format	Design Choice
candidate_id, full_name	string; string null	ID is deterministic from email, GitHub username, name, or source key. Name is trimmed only, not guessed.
emails, phones	string[]; E.164 phones	Emails are lowercase and valid-looking. Phones use library parsing; invalid numbers are dropped.
location, links	{city, region, country}; link object	Country becomes ISO-3166 alpha-2. Known links are separated; unknown useful links stay in other[].
headline, years_experience	string null; number null	Accepted from source fields or conservative note extraction only.
skills	{(name, confidence, sources[])}	Aliases and close typos map to canonical names, e.g. ml/machinelearning -> machine learning, k8s -> kubernetes.
experience, education	arrays	Experience dates normalize to YYYY-MM. Education remains empty unless a trusted source provides it.
provenance, conflicts, confidence	audit metadata	Shows where each value came from, which values disagreed, and how confident the result is.

Merge, Conflict Resolution, And Confidence

- Matching:** merge by exact normalized email first, then GitHub username, phone, and normalized full name. Weak matches stay separate unless the demo data has exactly one CSV and one GitHub record.
- Winner policy:** if two sources agree, that value wins. Otherwise structured sources beat free text, then higher confidence wins, then stable source order breaks ties.
- Lists:** emails, phones, skills, links, and experience are deduplicated so useful non-conflicting evidence is not lost.
- Conflicts:** losing scalar values are still saved with value, source, method, confidence, and whether they were chosen.
- Confidence:** source reliability plus cross-source agreement gives field confidence; overall confidence is the average. Shared email gives the strongest match confidence.

Runtime Custom Output Config

- The internal canonical profile is never changed for a client-specific output shape. A separate projection layer handles reshaping.
- fields can select fields, rename fields, or map from paths like emails[0] and skills[].name.
- Config can apply final normalization, for example E.164 for phones or canonical names for skills.
- include_confidence toggles confidence/provenance metadata in the projected response.
- on_missing supports null, omit, or error. Type checks ensure the custom schema is valid.

Edge Cases And What I Would Leave Out

- Missing or empty source:** keep running with available evidence; output nulls or empty arrays based on schema/config.
- Malformed values:** bad emails, impossible phones, unknown countries, and unparseable dates are removed rather than repaired by guessing.
- Duplicate candidate risk:** clear exact keys merge; ambiguous same-name candidates remain separate to avoid polluting a profile.
- Conflicting contact or title:** choose the safest winner, preserve the rejected value in conflicts, and lower confidence if evidence is weak.
- GitHub/API failure:** rate limit or lookup failure should not crash the run; GitHub simply contributes no values for that candidate.
- Out of scope under time pressure:** full PDF/DOCX resume NLP, fuzzy identity matching, inferred education, and ML ranking. These need more review data to be safe.

Validation approach: run unit tests for normalizer, notes extractor, merger, and projector; then run the CLI end-to-end on sample inputs for the default schema and one custom config output. The UI remains a thin input/output surface; correctness lives in the Python transformer.